MLFF PERF-P5: TRAIN2/EVAL2 Persistence and Reuse Hardening

mdstats development

2026-08-15

1. Status

Release: `mdstats 0.20.187a0`
Architecture: revision 54
Authority class: execution-only unless an evidence schema is explicitly versioned
GPU qualification: deferred to `FINAL-GPU1`

PERF-P5 reduces late TRAIN2/EVAL2 persistence overhead without changing restart state, checkpoint identity, evaluation metrics, checkpoint ranking, or downstream scientific decisions.

2. TRAIN2 streamed tensor hashing

The pre-P5 digest path converted every contiguous CPU tensor into another complete `bytes` object through `numpy().tobytes()`. The digest itself was correct, but the conversion created an avoidable allocation approximately equal to the tensor payload.

PERF-P5 keeps the existing digest byte contract and replaces only the transport into SHA-256. A detached contiguous CPU tensor is exported through Python's buffer protocol and fed to the hasher in bounded chunks. Python documents `memoryview` as a zero-copy view over objects exposing the buffer protocol.[1]

For tensor sequence $T = (t_1, \ldots, t_n)$, the scientific digest remains

$$H = \mathrm{SHA256}\left(S \Big\| \Big\|_i \left[D_i \| Q_i \| B_i\right]\right),$$

where S is the existing schema marker, D_i is dtype metadata, Q_i is shape metadata, and B_i is the exact contiguous CPU byte sequence. PERF-P5 changes neither D_i, Q_i, nor B_i.

The same buffer-streaming primitive is used by STOR2 evaluation-state capsule hashing. Historical capsule and TRAIN2 digest values therefore remain byte-identical.

2.1 Persistence telemetry

TRAIN2 now writes execution-only `train2_persistence.jsonl`. Each durable epoch records:

- live/EMA clone time;
- tensor-state hash time;
- raw-checkpoint hash time;
- continuation-companion write time;
- summary write time;
- total persistence time; and
- raw-checkpoint and companion byte counts.

This telemetry does not participate in TRAIN2 scientific or continuation digests.

[1] Python documentation, *Built-in Types - memoryview*: https://docs.python.org/3/library/stdtypes.html#memory-views.

No state was removed from `train2_runtime.pt`. Exact model, EMA, optimizer-reference, LR, RNG, exposure, and raw-checkpoint ancestry remains unchanged.

3. EVAL2 model-shell reuse

PERF-P5 adds an **optional** same-architecture model-state reload interface. A reusable shell is admissible only when:

1. it was created from an unaccelerated, uncompiled candidate model;
2. it is not a source-foundation-bound provider;
3. source and resident model classes match exactly;
4. state keys match exactly;
5. every tensor shape and dtype matches exactly; and
6. `load_state_dict(..., strict=True)` succeeds.

PyTorch defines `state_dict`/`load_state_dict` as the standard module-state persistence interface; strict loading requires matching state keys.[2]

The shell is mutable execution state and must never be shared across concurrent inference workers. The normal full-reconstruction path remains the default and exact fallback.

On the CPU development host, shell reload was **6.49% slower** than fresh `MACECalculator` construction for the supplied MH-1 model (median 105.28 ms versus 98.86 ms, three paired repetitions), although predictions were byte-identical. Therefore shell reuse is implemented but **not promoted as a CPU default**. It may be reconsidered only in `FINAL-GPU1`, where avoiding repeated accelerator conversion could change the cost balance.

4. DATA8 and dataset-format audit

PERF-P5 did not add an HDF5/LMDB conversion. MACE supports HDF5 and LMDB as training-data formats,[3] but a storage format is not, by itself, proof of a reusable authenticated `AtomicData` graph cache. Existing DATA8 fixed-file caching and DATA6/EVAL2 graph caches remain the qualified reuse mechanisms.

A source audit found no remaining DATA8 large-array authority that justified a new schema solely for PERF-P5. Existing fixed-file content-addressed reuse and native array persistence are retained.

5. CPU qualification

The development host exposes an AMD EPYC 9V74 CPU, 8-core cgroup quota, 4 GiB memory limit, Python 3.13.5, and PyTorch 2.10.0+cpu.

A 256,000,000-byte contiguous FP32 state was hashed in fresh processes using the exact pre-P5 and PERF-P5 byte contracts. Two independent samples per path give:

Path	Legacy median	PERF-P5 median	Wall change	Legacy peak-RSS increment	PERF-P5 peak-RSS increment
TRAIN2 state digest	265.02 ms	142.97 ms	**46.05% lower**	245.00 MiB	0.75 MiB
STOR2 capsule digest	248.19 ms	146.87 ms	**40.82% lower**	244.94 MiB	0.81 MiB

[2] PyTorch documentation, *Saving and Loading Models* and `Module.load_state_dict`: https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html; https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html#torch.nn.Module.load_state_dict.

[3] MACE documentation, *Heterogeneous Data Training* and *Large Dataset Pre-processing*: https://mace-docs.readthedocs.io/en/latest/guide/heterogeneous_data.html; https://mace-docs.readthedocs.io/en/latest/guide/multipreprocessing.html.

The old/new TRAIN2 digest is exactly

`c5e22dcc6fd8646fee9c6bdce424d59029abd5dbb286b0e353fceea3ec5568ca`.

The old/new capsule digest is exactly

`bd118d11da1689dd6b8f0c9a865b85a83cbf188686f9a54ee028430f1af716ad`.

The peak-RSS result is the direct consequence of removing the extra full-size `bytes` materialization; the tensor itself is present in both paths.

6. Acceptance and deferred work

PERF-P5 passes CPU/control-plane qualification when:

- legacy and streamed tensor digests are byte-identical;
- TRAIN2 pause/resume state remains complete;
- STOR2 capsules reconstruct the same deployable model state;
- optional shell loading fails closed on class/key/shape/dtype mismatch;
- prediction/evaluation caches retain their prior identities;
- no dataset-format substitution is represented as a graph cache; and
- measured persistence overhead decreases on the bounded CPU benchmark.

The release-matched GPU campaign still owns any claim that shell reuse, checkpoint persistence overlap, or accelerator-side checkpoint handling improves end-to-end training/evaluation throughput. Those measurements remain part of `FINAL-GPU1` and `PERF-CERT1`.